

Session 4 Task

Task 1: Implement a full MPU6050 driver:

- Create a class for your library. The class constructor should contain the address of the mpu6050 as a parameter.
- Setup function that setups the wire library and setups the mpu6050. Make sure to set the I2C clock to 400khz. Use the function definition ``

```
MPU::setup(const uint8_t address);
```

- Create the following functions to configure the MPU6050 sensors

```
void MPU::configure_gyro(const uint_t range); //  
confiures the range of the gyroscope  
void MPU::configure_acc(const uint_t range)  
//configures the range from the datasheet  
<typename T> MPU::get_gyro(); //function returns values  
of gyroscope, you can return them as a struct or send  
adress of each value as a parameter  
<typename T> MPU::get_acc(); //function returns values  
of accelerometer, you can return them as a struct or  
send adress of each value as a parameter
```

- Using the above functions: create a `void loop()` that calculates the yaw angle of the sensor (heading).
- implement `float MPU::get_pitch();` and `float MPU::get_roll()` that returns pitch and roll of the sensor. You can use the accelerometer values to obtain those angles. search the internet for the formulae.

Make sure to encapsulate everything within your class, create a .h and .cpp file for those.

Task 2: FreeRTOS

On a single Wokwi project create:

- Create task blinks and LED every 500 ms. Use `vTaskDelayUntil()` function to ensure accurate timing.
- Create an RTOS queue and 3 tasks:
 - the first task is responsible for dequeuing to the serial monitor. Make sure to only dequeue when the queue is not empty.
 - The second task should enqueue a string "Task one is working".
 - the third task should enqueue the string "Task two is working".
 - Only the first task should print to the serial monitor.